MySQL

select 查询.

基础查询 { select * from 表名; 查询所有列
select 列1, 列2 from 表名; 查询指定列

条件查询 { select * from student where age >= 18; 查询年龄>=18的在student表中.

常用条件

= 等于
!= 不等于
> < >= <=

between ... and 在区间内
in (值1, 值2) 在列表里.
like 'xxx' 模糊查询.
is null / is not null

模糊查询 like

where name like '张%' 以张开头的字符串 name

where name like '%明%' name中含有明

排序 order by

order by age ASC; 按年龄升序 ASC可不写 默认是升序
order by age DESC; 降序

分页 limit

select * from student limit 0,10; 从第0条开始取10条

二、聚合函数（统计查询）

count(*) 总条数
sum(列) 求和
avg(列) 平均值
max(列) 最大
min(列) 最小

例：
select count(*) from student
select avg(age) from student

三、分组查询 group by + having

select gender, count(*) → 统计每组人数
from student
group by gender; → 按性别分组.

group by gender
having count(*) > 5 → 分组后过滤

1. 多表查询

1. 内连接 inner join，只返回两边都匹配的数据.

select s.name, c.cname
from student s
inner join class c
on s.class_id = c.id

学生表中的班级id 匹配 班级表中的班级id

2、左连接 left join 左边表全部显示，右边没有就显示 null

```
select s.name, c.cname
from students
left join class c
on s.class_id = c.id
```

3、右连接 right join 右边表全部显示，左边没有显示 null

五、增删改

1、增加 insert

```
insert into student(name, age) values('张三', 18);
```

2、修改 update

```
update student
set age = 19
where id = 1
```

3、删除 delete

```
delete from student where id = 1;
```

六、约束

primary key 主键(唯一、非空)

not null 非空.

unique 唯一

default 默认值.

foreign key 外键

七、索引 ?

1. 索引是提高查询速度的数据结构(底层一般是B+树)

2. 什么时候建索引 查询频繁的列

3. 什么时候不要建：数据太少 经常增删改，区分度低

4. 创建索引

```
create index idx-name on student(name);
```

八、事务.

1. 事务是：一组操作，要么全部成功、要么全部失败.

2. 四大特性ACID

A. 原子性，不可分割

C 一致性，数据不混乱

I、隔离性：互不干扰

D. 持久性：提交后永久失效

3. 事务命令.

start transaction

update ...;

commit;

rollback

排序、分页

student表

id	name	age	score	gender
1	张三	19	85	男
2	李四	20	92	男
3	王五	19	78	男
4	赵六	21	95	女
5	钱七	20	88	女

按分数排序：select * from student order by score;

按年龄排序：select * from student order by age;

多字段排序：

先按年龄升序，年龄一样按分数降序

select * from student order by age ASC, ~~score~~ score DESC;

排序+分页

取分数最高的前三名.（先按分数降序排序，再取前3条）

select * from student order by score DESC limit 3;

聚合函数 + group by 分组

仍是 student 表

1、统计一共有多少学生

```
select count(*) from student
```

2、统计有多少男生

```
select count(*) from student where genden = '男';
```

3、求所有人的总分

```
select sum(score) from student.
```

4、一班学生的总分

```
select sum(score) from student
where class_id = 1;
```

一次查多个聚合函数

```
select
    count(*) AS 总人数,
    sum(score),
    avg(score)
from student
```

→ AS 是把展示的表头 count(*) 换成可读的"总人数"

group by + having 过滤

只展示人数大于等于2的班级id和总人数

```
select class_id,
count(*)
from student.
group by class_id
having count(*) >=2
```

只展示人数大于等于2的班级id.

```
select class_id
from student
group by class_id
having count(*) >=2
```

聚合函数 + group by 分组

按班级分组统计：每个班多少人，平均分，最高分.

```
select class_id, count(*), avg(score), max(score)
from student
group by class_id;
```

展示：

class_id	人数
1	3
2	2

多表查询

student

id	name	class_id
1	张三	1
2	李四	1
3	王五	1
4	赵六	2
5	钱七	Null

class

id	cname
1	计科1班
2	计科2班
3	计科3班

内连接 inner join

```
select
    s.id
    s.name
    c.cname
from student s
inner join class c
on s.class-id = c.id.
```

展示:

id	name	cname
1	张三	计科1班
2	李四	计科1班
3	王五	计科1班
4	赵六	计科2班

左连接

左边表全部显示，右边表匹配不到的显示NULL

```
select
    s.id
    s.name
    c.cname
from student s
left join class c
on  s.class_id = c.id
```

展示：

id	name	cname
1	张三	计科1班
2	李四	计科1班
3	王五	计科1班
4	赵六	计科2班
5	钱七	NULL

左边student表全部显示

右边表匹配不上就显示NULL

右连接 right join

```
select
    s.id
    s.name
    c.cname
from student s
right join class c
on s.class_id = c.id
```

展示：

id	name	cname
1	张三	计科1班
2	李四	计科1班
3	王五	计科1班
4	赵六	计科2班
NULL	null	计科3班

左边表没有匹配的NULL

右边表全部显示

① 创建表.

```
create table student {
    id int primary key auto_increment,      主键自增
    name varchar(20) not null,              姓名不为空.
    age int,
    gender char(1)
    class_id int.
}
```

② 修改表

添加一列 add

alter table student add score int; 给 student 表加一列 score.

删除一列 drop

alter table student drop gender;

③ 修改列的类型/名字 modify / change

- 修改列类型 modify

alter table student modify name varchar(30);

修改列名 change

alter table student change age stu_age int;

④ 修改表名. rename

alter table student rename to stu;

⑤ 删除表 drop table student

索引 表中建了个B+树 MySQL会[illegible]
B+树只有索引列和真实的指针
加了索引会从表头一层一层往下
变成直接走索引 瞬间找到

索引 = 书的目录
平常没有目录 需要从头翻到尾
有目录 直接快速定位了.
(用空间换时间)

适合加索引的列：
① 经常出现在 where 里的字段
② 经常用于 join 关联的字段
③ 经常 order by / group by 的字段

如何创建索引 ① 普通索引
给经常查询的 name 加索引
create index idx_name on student(name);

② 唯一索引 (值不能重复)
给手机，身份证号使用
create unique index idx_phone on student(phone);

③ 主键索引 (自带，不用创建)
如 id.

④ 联合索引 (一次给多个字段建索引)
如经常按 class_id + score 查询.
create index idx_class_score on student(class_id, score);

联合索引，最左匹配原则
创建了联合索引后
如果 select * from student where class_id = 1
那么命中了索引，可以走索引查询
如果 where class_id = 1 and score = 90
(反过来写也命中)
那么命中了索引 可以走索引查询
如果只有 where score = 90
那么无法命中.
因为 class_id 是最左

事务

山东大学

要么全成功，要么全失败

① 转账、 account {
id int,
name varchar(20)
} money int 存款

数据 张三 1000元
李四 1000元

要求 张三 转给李四 200元 { 张三 -200 不能只减不加
李四 +200 不能只加不减

start transaction; 开启事务

update account set money = money -200 where name = '张三'; 张三扣钱

update account set money = money +200 where name = '李四'; 李四加钱

commit; 没有问题，提交

// rollback; (如果出错 则写上) 回滚.